

Лабораторная работа №4

Введение в OpenMP. Простые примеры параллельных программ

Задание:

1. Изучить примеры 1.1-1.5.
2. Откомпилировать и запустить программы, используя 2 потока.
3. Используя выводимую на экран информацию убедиться в том, что выполняются 2 потока, и при этом обеспечивается требуемая синхронизация

ПРИМЕР 1.1

Каждый поток (ветвь параллельной-программы) выводит на экран свой идентификационный номер и количество заказанных параллельных процессов.

```
#include <omp.h>
#include <stdio.h>

int main ()
{
    int size, rank;

    /* Создание множества параллельных процессов и в каждом из них задаются
     * свои приватные переменные size и rank */
    #pragma omp parallel private(size, rank)
    {

        /* Каждый процесс находит свой порядковый номер и выводит его на экран */
        rank = omp_get_thread_num();
        printf("Hello World from thread = %d\n", rank);

        /* Главный процесс - master выводит на экран количество процессов */
        if (rank == 0)
        {
            size = omp_get_num_threads();
            printf("Number of threads = %d\n", size);
        }

    } /* Завершение параллельной части */
}
```

ПРИМЕР 1.2

Параллельное суммирование элементов двух векторов. Суммирование осуществляется в цикле. В программе применяется комбинация параллельного цикла и редуцированной операции по всем процессам.

```
#include <omp.h>
#include <stdio.h>

int main ()
{
    int i, n;
    float a[100], b[100], sum;

    /* Инициализация элементов векторов */
    n = 100;
    for (i=0; i < n; i++)
        a[i] = b[i] = i * 1.0;
    sum = 0.0;

    /* Создание множества параллельных процессов и распараллеливание
```

```

* цикла по виткам. При выходе из цикла все значения переменной sum
* суммируются по всем процессам. */
#pragma omp parallel for reduction(+:sum)
for (i=0; i < n; i++)
    sum = sum + (a[i] * b[i]);

/* Главный процесс выводит на экран значение sum */
printf("    Sum = %f\n",sum);
}

```

ПРИМЕР 1.3

Пример аналогичный предыдущему: Параллельное суммирование элементов двух векторов. Но суммирование осуществляется в цикле в отдельной подпрограмме. В программе применяется комбинация параллельного цикла и редуцированной операции по всем процессам.

```

#include <omp.h>
#include<stdio.h>

#define VECLLEN 100
float a[VECLLEN], b[VECLLEN], sum;

/* Подпрограмма, в которой суммируются элементы векторов */
float dotprod ()
{
    int i,rank;

    rank = omp_get_thread_num();
    #pragma omp for reduction(+:sum)
    for (i=0; i < VECLLEN; i++)
    {
        sum = sum + (a[i]*b[i]);
        printf(" rank = %d i=%d\n",rank,i);
    }
    return(sum);
}

main ()
{
    int i;

    /* Инициализация элементов векторов */
    for (i=0; i < VECLLEN; i++)
        a[i] = b[i] = 1.0 * i;
    sum = 0.0;

    /* Создание множества параллельных процессов */
    #pragma omp parallel
        sum = dotprod();

    printf("Sum = %f\n",sum);
}

```

ПРИМЕР 1.4

Пример аналогичный примеру 3.2: Параллельное суммирование элементов двух векторов. Но суммирование осуществляется в отдельных секциях.

```

#include <omp.h>
#include<stdio.h>

```

```

#define N      50

int main ()
{
    int i, size, rank;
    float a[N], b[N], c[N];

    /* Инициализация элементов векторов */
    for (i=0; i < N; i++)
        a[i] = b[i] = i * 1.0;

    /* Создание множества параллельных процессов */
    #pragma omp parallel shared(a,b,c) private(i,rank,size)
    {

        /* Каждый процесс находит свой порядковый номер и выводит его на экран */
        rank = omp_get_thread_num();
        printf("Thread %d starting...\n",rank);

        /* Директива задания секций */
        #pragma omp sections nowait
        {
            /* Секция 0*/
            #pragma omp section
            for (i=0; i < N/2; i++)
            {
                c[i] = a[i] + b[i];
                printf("rank = %d i= %d c[i]= %f\n", rank,i,c[i]);
            }
            /* Секция 1*/
            #pragma omp section
            for (i=N/2; i < N; i++)
            {
                c[i] = a[i] + b[i];
                printf("rank = %d i= %d c[i]= %f\n", rank,i,c[i]);
            }

            } /* Завершение блока секций */

        if (rank == 0)
        {
            size = omp_get_num_threads();
            printf("Number of threads = %d\n", size);
        }

    } /* Завершение параллельной части */
}

```

ПРИМЕР 1.5

Пример параллельного умножения матрицы на вектор.

```

#include <omp.h>
#include<stdio.h>

#define M 10

int main ()
{
    float A[M][M], b[M], c[M];
    int i, j, rank;

```

```

/* Инициализация данных */
for (i=0; i < M; i++)
{
    for (j=0; j < M; j++)
        A[i][j] = (j+1) * 1.0;
    b[i] = 1.0 * (i+1);
    c[i] = 0.0;
}
printf("\nВывод значений матрицы A и вектора b на экран:\n");
for (i=0; i < M; i++)
{
    printf("  A[%d]= ", i);
    for (j=0; j < M; j++)
        printf("%.1f ", A[i][j]);
    printf("  b[%d]= %.1f\n", i, b[i]);
}

/* Создание множества параллельных процессов и в каждом из них задаются
 * свои приватные переменные rank и i*/
#pragma omp parallel shared(A,b,c,total) private(rank,i)
{
    rank = omp_get_thread_num();

    /* Директива распараллеливания цикла по виткам */
    #pragma omp for private(j)
    for (i=0; i < M; i++)
    {
        for (j=0; j < M; j++)
            c[i] += (A[i][j] * b[j]);
    }

    /* Каждый процесс выводит свой порядковый номер, значение витка цикла и
     * значение результирующего вектора на каждом витке цикла и внутри
     * критической секции */
    #pragma omp critical
    {
        printf(" rank= %d i= %d c[%d]=%.2f\n", rank, i, c[i]);
    }
} /* Конец параллельного цикла */
} /* Завершение параллельной конструкции */
}

```